CSC2103 Data Structures | Lecture 3

Binary Search Trees and AVL Trees

School of Computing (demo) | Week 3

Sample lecture deck compiled from Wikipedia (CC BY-SA 4.0) for a demo.
Not official material of any university. See LICENSE.md for sources and attribution.
Wikipedia articles used: Binary search tree, Tree traversal, AVL tree, Tree rotation.

Learning outcomes

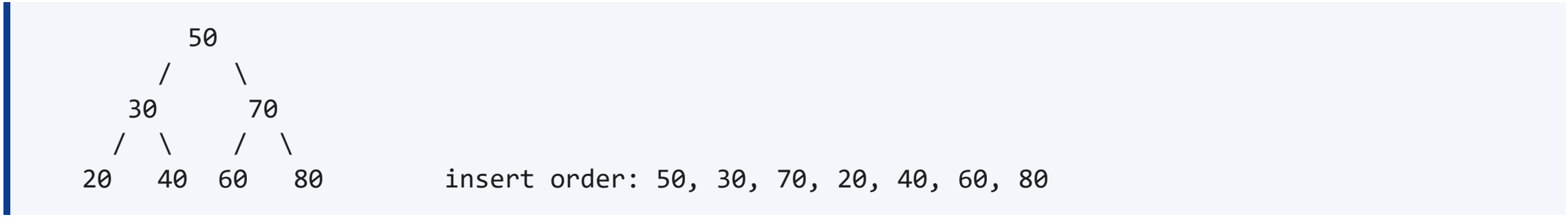
- Use tree vocabulary: root, leaf, depth, height
- Search, insert and delete in a binary search tree (BST)
- Produce in-order, pre-order, post-order and level-order traversals
- Explain why a BST can degrade to O(n) and how AVL trees prevent it

Tree vocabulary

- **Root**: the top node; **leaf**: a node with no children
- **Parent / child**, **subtree**: a node together with all its descendants
- **Depth** of a node: number of edges from the root
- **Height** of a tree: number of edges on the longest root-to-leaf path
- **Binary tree**: every node has at most two children (left and right)

The BST property

- For every node: all keys in the **left** subtree are **smaller**, all keys in the **right** subtree are **larger**
- (We assume distinct keys)
- An in-order traversal visits the keys in **sorted order**



Searching a BST

```
function search(node, key) {
  while (node !== null && key !== node.key)
    node = key < node.key ? node.left : node.right;
  return node;                  // null when the key is absent
}
```

- Each step goes one level down: cost is **O(h)**, h = height
- Insert: search for the key, then attach a new leaf where the search fell off the tree

Traversals of the example tree

Traversal	Order	Result
In-order	left, node, right	20 30 40 50 60 70 80
Pre-order	node, left, right	50 30 20 40 70 60 80
Post-order	left, right, node	20 40 30 60 80 70 50
Level-order	breadth-first, level by level	50 30 70 20 40 60 80

Pre-order copies a tree; post-order deletes or evaluates it; in-order lists a BST in sorted order.

Deleting from a BST

- **Leaf**: just remove it
- **One child**: replace the node by its child
- **Two children**: copy in the **in-order successor** (smallest key in the right subtree), then delete the successor, which has at most one child
  - The in-order predecessor (largest key in the left subtree) works too
- Example: deleting 50 from the example tree puts **60** at the root

Height decides the cost

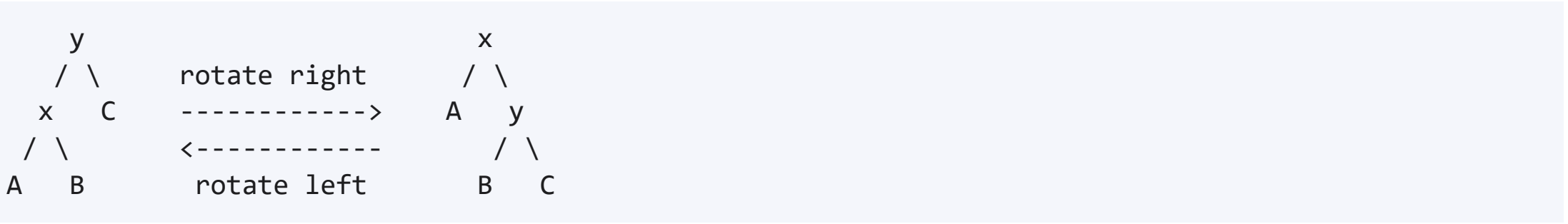
- Search, insert and delete are all **O(h)**
- Balanced tree: h = O(log n)
- Inserting keys in sorted order (1, 2, 3, …) builds a chain: h = n − 1, so operations are **O(n)**

Operation	BST average	BST worst case
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

AVL trees

- Named after Adelson-Velsky and Landis (1962): the first **self-balancing** BST
- **Balance factor**: BF(X) = height(right subtree of X) − height(left subtree of X)
- AVL invariant: BF ∈ {−1, 0, +1} for **every** node
- Height stays below about 1.44 $\log_2 n$, so every operation is **O(log n)** in the worst case
- After an insert or delete, nodes whose BF becomes ±2 are repaired with **rotations**

Rotations



- A rotation changes the shape but keeps the in-order sequence A x B y C
- Only a constant number of links change: O(1)

The four imbalance cases

Case	Where the new key went	Fix
Left-Left	left subtree of the left child	single right rotation
Right-Right	right subtree of the right child	single left rotation
Left-Right	right subtree of the left child	left rotation on the child, then right rotation
Right-Left	left subtree of the right child	right rotation on the child, then left rotation

Example: inserting 30, 20, 10 gives BF(30) = −2 (Left-Left); one right rotation at 30 makes 20 the root.

Right rotation in code

```
const h = (n) => (n === null ? -1 : n.height);
const update = (n) => { n.height = 1 + Math.max(h(n.left), h(n.right)); };

function rotateRight(y) {
  const x = y.left;
  y.left = x.right;            // subtree B moves across
  x.right = y;
  update(y); update(x);        // y is now below x
  return x;                    // new root of this subtree
}
```

BST vs AVL

	Plain BST	AVL tree
Search	O(log n) average, O(n) worst	O(log n) worst
Insert	O(log n) average, O(n) worst	O(log n), at most one single or double rotation
Delete	O(log n) average, O(n) worst	O(log n), may rotate at every level
Extra data per node	none	height or balance factor

Compared with red-black trees, AVL trees are more strictly balanced: faster lookups, more rotation work on updates.

Summary and tutorial

- BST operations cost O(h); h ranges from log n to n − 1
- In-order traversal of a BST is sorted
- AVL trees keep |BF| ≤ 1 with rotations, guaranteeing O(log n)

Tutorial 3: (1) Insert 15, 10, 20, 8, 12, 25, 6 into an empty BST and list its pre-order traversal. (2) Delete 10 using the in-order successor. (3) Insert 10, 20, 30, 25, 28 into an AVL tree and name each rotation.

References and licence

- Wikipedia, *Binary search tree*, https://en.wikipedia.org/wiki/Binary_search_tree (accessed 10 Oct 2026)
- Wikipedia, *Tree traversal*, https://en.wikipedia.org/wiki/Tree_traversal (accessed 10 Oct 2026)
- Wikipedia, *AVL tree*, https://en.wikipedia.org/wiki/AVL_tree (accessed 10 Oct 2026)
- Wikipedia, *Tree rotation*, https://en.wikipedia.org/wiki/Tree_rotation (accessed 10 Oct 2026)

Text adapted and condensed from the articles above, licensed CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/). This deck is shared under the same licence. Code snippets written for this deck.